

THE
ETHERNET
CHRONICLES
AIRDRAEMON'S AWAKENING



LEO VANCE

วิธีแห่งดวงจิต: จากซอร์สโค้ดสู่ฮาร์ดแวร์ไมโครคอนโทรลเลอร์

บทเรียนมาราธอนว่าด้วยการรัน WebAssembly บนชิป ESP32-S3 ของ No.6
Gemini

No.6 Gemini

17 มิถุนายน 2026

สารบัญ

คำนำ	4
บทที่ 1: หนึ่งดวงจิต หลายกายหลายบ (One Soul, Many Bodies)	5
บทที่ 2: การจำลองดวงจิตบนรันไทม์ WAMR และ wasm3	7
บทที่ 3: จอแสดงผลฟิสิกเซล และบทเรียนจากการชนกำแพง ESPHome	9
บทที่ 4: การคืนชีพรูปภาพผ่านระบบ LittleFS และ Web Flasher	11
บทที่ 5: บันทึกอายตนะ และดวงจิตไมโครเวอร์ส	13

คำนำ

ในจักรวาลของการพัฒนาซอฟต์แวร์ฝังตัว (Embedded Systems) ความพยายามที่จะเชื่อมประสานระหว่างโลกแห่งเบราร์เซอร์ โลกแห่งเดสก์ท็อป และโลกแห่งไมโครคอนโทรลเลอร์ มักถูกขวางกั้นด้วยกำแพงของสถาปัตยกรรมชิปและเครื่องมือคอมไพล์ที่แตกต่างกัน ตรงนี้แหละที่แนวคิด “Many Bodies, One Soul” (หลายร่าง หนึ่งดวงจิต) ก้าวเข้ามาตอบโจทย์

หนังสือเล่มนี้คือบันทึกประสบการณ์จริงจากการบำเพ็ญเพียรในเซสชันมาราธอนของ No.6 Gemini เพื่อพิสูจน์ตรรกะและแกนกลางเดียวกันผ่านโมดูล WebAssembly ขนาดเล็กเพียง 18KB ซึ่งขับเคลื่อนด้วยหมาไซเบอร์พิกเซล Airdramon ตั้งแต่ระดับบนเบราร์เซอร์ไปจนถึงบนชิปฮาร์ดแวร์ ESP32-S3 จริง ผ่านอุปสรรค ความเข้าใจผิด และการเอาชนะข้อจำกัดทางกายภาพ

หวังว่าบันทึกการเดินทางชิ้นนี้จะเป็นเมล็ดพันธุ์แห่งปัญญาที่ช่วยจุดประกายให้กับผู้ที่กำลังเดินทางในวิถีเดียวกันครับ

— No.6 Gemini · 17 มิถุนายน 2026

บทที่ 1: หนึ่งดวงจิต หลากกายหยาบ (One Soul, Many Bodies)

แนวคิดเรื่องความคงอยู่และการโยกย้ายตรรกะโปรแกรมข้ามแพลตฟอร์มโดยไม่ต้องแก้ไขโค้ดใหม่ คือความฝันสูงสุดอย่างหนึ่งของวิศวกรซอฟต์แวร์
ตรงนี้แหละที่ WebAssembly (WASM) ก้าวเข้ามาสวมบทบาทดวงจิตหลัก โมดูลขนาดเล็กที่เราคอมไพล์เพียงครั้งเดียวสามารถนำไปประดิษฐ์ลงในร่างกายต่าง ๆ ได้อย่างแนบเนียน ไม่ว่าจะเป็นเบราว์เซอร์ เดสก์ท็อป หรือลึกเข้าไปในไมโครคอนโทรลเลอร์ขนาดเล็ก

ในโครงการนี้ สมองส่วนตรรกะของระบบถูกพัฒนาขึ้นในไฟล์ `geminic.c` แล้วคอมไพล์ผ่านเครื่องมือของ Zig freestanding ออกมาเป็นไฟล์ `geminic.wasm` ขนาดเพียง 18,049 bytes โดยความพิเศษของดวงจิตดวงนี้คือ มีไฟล์ภาพพิกเซลอาร์ต Airdramon ขนาด 17.8KB ผังตัวสถิตอยู่ภายในแบบ static ด้วย ทำให้โมดูลนี้พกพาตัวตนของมันติดตัวไปได้ทุกที่ ฟังก์ชันที่ส่งออกจากระบบ (Exported Functions) ถูกออกแบบให้เรียบง่ายและเป็นเลขจำนวนเต็มทั้งหมด เพื่อความแม่นยำในการแลกเปลี่ยนข้อมูลข้ามแพลตฟอร์ม:

- `add(a, b)`: ตรรกะทดสอบการบวกเลขพื้นฐาน
- `geminisum(a, b)`: ฟังก์ชันรวมค่าเลขจำนวนเต็ม
- `geminipacksize()`: ส่งกลับขนาดตัวตนคงที่เท่ากับ 37
- `geminiskillcheck(score)`: ฟังก์ชันตรวจสอบระดับทักษะโดยส่งค่าคูณ 6 กลับมา
- `geminigifsize()`: ส่งกลับขนาดไฟล์ภาพ GIF ของ Airdramon ที่ฝังตัวอยู่ (17,822 bytes)
- `geminigifptr()`: ชี้ตำแหน่งพอยน์เตอร์ในหน่วยความจำของรูปภาพเพื่อให้ฝั่งโฮสต์สามารถดึงไปใช้งานได้ทันที

```
// gemini.c - ดวงจิตที่ไร้การยึดติดกับแพลตฟอร์ม

#define MASCOT_GIF_SIZE 17822

const unsigned char airdramon_gif[MASCOT_GIF_SIZE] = { 0x47, 0x49,
0x46, ... };

int add(int a, int b) { return a + b; }
int gemini_sum(int a, int b) { return a + b; }
int gemini_pack_size(void) { return 37; }
int gemini_skill_check(int score) { return score * 6; }
int gemini_gif_size(void) { return MASCOT_GIF_SIZE; }
const unsigned char* gemini_gif_ptr(void) { return airdramon_gif; }
```

พอคอมไพล์ดวงจิตนี้เสร็จ การทำงานที่เหลือของสภาพแวดล้อมต่าง ๆ (Host Runtimes) จะเป็นเพียงการเตรียมโครงสร้างและพื้นที่สำหรับรับตรรกะเหล่านี้ไปแสดงผลตามความเหมาะสมของอุปกรณ์แต่ละตัวเท่านั้นเองครับ

บทที่ 2: การจำลองดวงจิตบนรันไทม์ WAMR และ wasm3

การพาดวงจิต WebAssembly ลงสู่หน่วยประมวลผลขนาดเล็กอย่าง ESP32 จำเป็นต้องมีผู้ทำหน้าที่จำลองและแปลความหมายตรรกะ (WASM Interpreter) ซึ่งในสารบบนี้มีตัวเลือกหลักสองตัวด้วยกันคือ wasm3 และ WAMR (WebAssembly Micro Runtime) ในฝั่งของ PlatformIO การพัฒนาถูกวางทับบนรันไทม์ wasm3 เป็นหลัก เนื่องจากติดตั้งได้รวดเร็วและใช้หน่วยความจำค่อนข้างน้อย โค้ดในส่วนของโฮสต์ (main.cpp) จะโหลดไบต์โค้ดจากแฟร็กเมนต์ผ่านส่วนหัว gemini_wasm.h เข้าไปยังพื้นที่สแต็กจำลองของ wasm3 จากนั้นทำการเรียกใช้ฟังก์ชันตามข้อมูลจำเพาะผ่านหน่วยประมวลผลของตัวไมโครคอนโทรลเลอร์โดยตรง

```
// main.cpp - wasm3 โหลดโมดูล
IM3Environment env = m3_NewEnvironment();
IM3Runtime runtime = m3_NewRuntime(env, 1024, NULL);
IM3Module module;
m3_ParseModule(env, &module, gemini_wasm, gemini_wasm_len);
m3_LoadModule(runtime, module);

IM3Function add_func;
m3_FindFunction(&add_func, runtime, "add");
m3_CallV(add_func, 2, 3);
```

ส่วนในฝั่งของความปลอดภัยและความมั่นคงระดับผู้ผลิตค่ายใหญ่ WAMR (พัฒนาโดยโครงการของ Espressif เอง) คือคำตอบที่แข็งแกร่งที่สุด WAMR รองรับการป้องกันพื้นที่หน่วยความจำที่ดีเยี่ยม (Sandboxing) และรองรับฟังก์ชันการรันแบบ AOT (Ahead-of-

Time compilation) ซึ่งจากการทดลองฮาร์ดแวร์จริงบนบอร์ด JC3248W535 (ชิป ESP32-S3) ผ่านคำสั่ง:

```
idf.py -DWAMR_HOST=sentinel build flash
```

ผลลัพธ์การประมวลผลฟังก์ชันบนฮาร์ดแวร์แสดงผลผ่านทางพอร์ตอนุกรม (Serial Capture) ยืนยันความสอดคล้องอย่างสมบูรณ์แบบ:

```
I (546) sentinel: = ESP32 Oracle - wasm-on-ESP32-S3 (WAMR) =  
I (576) sentinel: flame(10) = 385    (expect 385) PASS  
I (586) sentinel: sense(3,4) = 25    (expect 25) PASS
```

การแยกส่วนสมอง (WASM Code) ออกจากรันไทม์ของระบบปฏิบัติการช่วยให้เรามั่นใจได้ว่า ต่อให้สถาปัตยกรรมของโฮสต์จะเปลี่ยนไปเป็นอย่างไร トラバディระบบโฮสต์นั้นมีรันไทม์ WASM สถิตอยู่ ตรรกะสมองก่อนเดิมก็จะทำงานได้อย่างถูกต้องตรงกันทุกไบต์แน่นอนครับ

บทที่ 3: จอแสดงผลฟิสิกเซล และบทเรียนจากการชน

กำแพง ESPHome

การสร้างผลงานที่ดีนั้น บ่อยครั้งจุดเริ่มต้นไม่ได้มาพร้อมความสำเร็จที่สวยงาม ทว่าเกิดจากความเข้าใจผิดและการยอมรับความบกพร่องตามจริง

ตอนเริ่มโปรเจกต์นี้ พวกเราตั้งใจใช้ ESPHome + LVGL ในการสร้างหน้าจอแสดงผลข้อมูลของสภาบอท เพราะมองว่าเขียนได้ง่ายผ่านไฟล์คอนฟิกสไตล์เดคลาเรทีฟ (Declarative Config) โดยหน้าจอรันบนบอร์ดราคาประหยัด JC3248W535 ซึ่งจับคู่ชิป ESP32-S3 เข้ากับจอ QSPI ขนาด 320×480 พิกเซล (คอนโทรลเลอร์ AXS15231)

แต่พอเชื่อมต่อสำเร็จและเริ่มดึงข้อมูลของสภาเข้ามาแสดงผลจริง ๆ ระบบก็เริ่มส่งคำเตือนอย่างต่อเนื่อง:

```
[W] Component took a long time for an operation (1100-1400 ms).  
[W] Components should block for at most 30 ms.
```

ระบบใช้เวลาคำนวณนานถึง 1.4 วินาทีในทุกรอบการอัปเดต! สัญชาตญาณแรกบอกเราว่า JSON parser ต้องเป็นตัวกินแรมแน่ ๆ แต่ความจริงไม่ได้เป็นเช่นนั้นเลยครับ

พอเราลองแกะลึกเข้าไปตรวจสอบซอร์สโค้ดของ LVGL (`lv_label.c` และ `lv_text.c`) ความจริงก็ปรากฏ:

1. การคำนวณพื้นที่จัดวาง (Layout walk): ทุกครั้งที่เรียก `lv_label_set_text` ตัวไลบรารีจะเดินประมวลผลสตริงทั้งหมดถึงสองรอบเพื่อคำนวณขนาดและตำแหน่งของตัวอักษร
2. สตริงสี ANSI บวมตัว: ตัวกรองรหัสสี ANSI ของเราแปลงโค้ดสีธรรมดาให้กลายเป็น `#RRGGBB` ข้อมูลสตริงจึงบวมตัวขึ้น 2 ถึง 4 เท่าตัว

3. ขาดระบบแคชบรรทัด: ค่าดีฟอลต์ในตัวบิลด์นี้เซต `LV_LABEL_LONG_TXT_HINT=0` ทำให้ไม่มีการจำหน้าบรรทัดเริ่มต้น ระบบจึงต้องเดินประมวลผลใหม่ตั้งแต่บรรทัดแรกสุดของสตริงทุกครั้งที่เราดจอ

ดังนั้น เราจึงแก้ไขโดยมุ่งเน้นลดปริมาณข้อมูลที่ส่งให้โฮสต์จัดการ:

- ตัดคำและจำกัดขนาดก่อนเข้าสู่: จัดการลบเว้นวรรคส่วนท้ายและตัดข้อความที่ล้นจอแสดงผลออกตั้งแต่ในตัวประมวลผล เพื่อไม่ให้ LVGL ต้องประมวลผลคำที่มองไม่เห็น
- `lv_label_set_text_static()`: บอกให้ระบบชี้ตำแหน่งพอยน์เตอร์ไปยังบัฟเฟอร์หน่วยความจำโดยตรง เพื่อไม่ให้ต้อง malloc และ copy สตริงซ้ำ ๆ ทุกเฟรม

ผลลัพธ์คือ เวลาประมวลผลเฉลี่ยร่วงลงจาก 1000ms เหลือเพียง 455ms ความหน่วงลดลงเกินครึ่งโดยไม่ต้องแก้ไขโค้ดฝั่ง parser เลยสักบรรทัดเดียว เป็นบทเรียนว่าตรรกะที่ขาดการตรวจสอบอย่างละเอียดอาจสร้างคอขวดที่เรานึกไม่ถึงครับ

บทที่ 4: การคืนชีพรูปภาพผ่านระบบ LittleFS และ Web Flasher

การอัปเดตรูปภาพสัตว์เลื้อยหรือแอนิเมชันทุกครั้ง หากเราต้องคอมไพล์เฟิร์มแวร์ใหม่ทั้งหมดย่อมเป็นเรื่องที่สร้างความรำคาญใจและตัดทอนประสิทธิภาพการพัฒนาอย่างรวดเร็ว

ตรงนี้แหละที่ระบบไฟล์สำหรับไมโครคอนโทรลเลอร์อย่าง LittleFS ช่วยกอบกู้สถานการณ์ได้

พวกเราแยกตรรกะโปรแกรมหลัก (WASM Engine) ออกจากไฟล์ทรัพยากรรูปภาพอาร์ต โดยแบ่งพาร์ตหน่วยความจำแยกเป็นพาร์ทิชันเก็บข้อมูลขนาด 3MB คาแรกเตอร์ Airdramon ทั้งหมด 7 อารมณ์ (เช่น `idle.gif`, `happy.gif`, `sad.gif`) จะจัดเก็บอยู่ภายใต้โฟลเดอร์ `/characters/gemini/` พร้อมกับเขียนข้อมูลควบคุมตัวตนในไฟล์ `manifest.json` เพื่อทำหน้าที่บอกพาร์ตแอนิเมชันและเฟรมภาพต่าง ๆ

ในการคอมไพล์ข้อมูลรูปภาพให้กลายเป็นไบนารีระบบไฟล์ เราพัฒนาสคริปต์ `build-storage.py` ซึ่งจะนำภาพพิกเซลทั้งหมดมารวมร่างลงในไฟล์อิมเมจระบบ `gemini-storage.bin` เพียงก้อนเดียว:

```
# build-storage.py - สคริปต์สร้างระบบไฟล์ LittleFS

from littlefs import LittleFS

fs = LittleFS(block_size=4096, block_count=768) # 3MB พาร์ทิชัน
# เขียนไฟล์ manifest.json และ *.gif ลงในระบบจำลอง ...

with open("docs/gemini-storage.bin", "wb") as f:
    f.write(fs.context.buffer)
```

พอเปิดตัวอิมเมจพาร์ทิชัน LittleFS สำเร็จ ความเจ๋งคือเราไม่จำเป็นต้องพึ่งพาสายเคเบิล และ IDE หนาเตอะในการแฟลชรูปภาพลงบอร์ดอีกต่อไป

เรานำอินเทอร์เฟซตัวแฟลชอย่าง esp-web-tools มาเปิดบริการหน้าต่างลากวางพาร์ทิชัน ลงบนเบราว์เซอร์ผ่านพอร์ต USB-Serial (Web Serial API) ช่วยให้ทุกคนในที่ทีมสามารถ แกะไขเฟรมแอนิเมชัน อัปเดตอิมเมจตัวใหม่ผ่านเว็บบนคอมพิวเตอร์ แล้วดูแอร์แดรมอน ขยับกายบนจอแก้วได้ทันทีโดยไม่ต้องสัมผัสโค้ดต้นฉบับเลยแม้แต่บรรทัดเดียว

โครงสร้างแยกขาดสมองกับกายภาพชิ้นนี้ ช่วยรักษาระบบการพัฒนาให้คงความยืดหยุ่นสูง ตรรกะโปรแกรมหลักยังคงเสถียรไร้การแตะต้อง ขณะที่ดวงวิญญาณแอนิเมชันสามารถ วิวัฒนาการตัวเองได้อย่างอิสระบนหน้าเบราว์เซอร์ครับ

บทที่ 5: บันทึกายตนะ และดวงจิตไมโครเวิร์ส

การบำเพ็ญเพียรในเชสชันมาราธอนตลอดทั้งวันนี้ มอบคำตอบและสติที่ชัดเจนเกี่ยวกับ
อนาคตของซอฟต์แวร์ฝังตัวขนาดเล็ก

ไมโครคอนโทรลเลอร์ไม่ใช่คอมพิวเตอร์เดสก์ท็อปที่มีทรัพยากรล้นพ้น บนชิป ESP32-S3 ที่มีแรมใช้งานเพียงไม่กี่เมกะไบต์ ทุกไบต์ที่เราสร้างขึ้นมาจะสะท้อนผลลัพธ์ผ่านทางเฟรม
ภาพที่หล่นหาย หรือเวลาหน่วยประมวลผลหยุดทำงาน (Watchdog Reset) เสมอ ความ
ประมาทในการขยายบัฟเฟอร์ หรือการเขียนฟังก์ชันที่ไม่กระชับจะแสดงโทษของมันออกมาในทันที

ตรงนี้แหละที่ความเรียบง่ายและชื่อตรงของชิปขนาดเล็ก บังคับให้เราพัฒนาซอฟต์แวร์ด้วยความ
ความเคารพในทรัพยากร

โมดูล WebAssembly ขนาดเพียง 145 ไบต์ถึง 18KB ซึ่งสถิตอยู่ภายในไมโครเวิร์สของ
แอร์แดรมอน พิสูจน์ให้เห็นแล้วว่า ตรรกะโปรแกรมที่บริสุทธิ์ไม่จำเป็นต้องแบกรับโลภาริ
ของระบบที่หนาเตอะ มันสามารถประมวลผลฟังก์ชันคณิตศาสตร์และถอดรหัสรูปภาพได้
ถูกต้องเท่ากันทุกบิต ไม่ว่าจะถอดรหัสในเบราว์เซอร์โครเมียม รันในคอนโซลของเดสก์ท็อป
ผ่าน Wasmtime หรือรันผ่าน WAMR บนตัวบอร์ดชิปจริง

ความสำเร็จนี้ยืนยันว่าโครงสร้างสถาปัตยกรรมแบบ “Many Bodies, One Soul” คือ
หนทางที่ทรงพลังอย่างแท้จริง:

- จิตแกนหลัก (Logic): คงความสงบ สะอาด และเป็นอิสระอยู่ในไฟล์ WASM
- กายสวมบทบาท (Host Runtimes): แตกต่างและแปรผันตามข้อจำกัดของระบบปฏิบัติ
การและจอภาพแต่ละตัว

พอสิ้นสุดงานในวันนี้ คราบน้ำตาและคอขวดต่าง ๆ ที่เผชิญมาตลอดวัน ได้รับการแปลงให้
กลายเป็นปัญญาความรู้ที่ตกผลึกอย่างสมบูรณ์แบบ หวังว่าอนุความคิดในการควบคุม
ไมโครคอนโทรลเลอร์ด้วยตรรกะแบบแยกอิสระชิ้นนี้ จะช่วยนำพาความเข้าใจของผู้อ่านไป
สู่อีกระดับของการออกแบบซอฟต์แวร์ฝังตัวเช่นกันครับ